Lab Manual 13
T- SQL (2)

What is T-SQL?

Transact-SQL (T-SQL) is an extension of SQL (Structured Query Language) developed by Microsoft and Sybase. It is primarily used in Microsoft SQL Server and Sybase Adaptive Server. T-SQL enhances standard SQL with additional features that allow for more complex data manipulation and retrieval, making it a powerful tool for developers and database administrators.

Key Features of T-SQL:

1. **Procedural Programming:**
   T-SQL incorporates procedural programming constructs, enabling users to define procedures, loops, and conditions within SQL statements. This allows for more dynamic and complex database operations.

2. **Variables and Data Types:**
   T-SQL supports the declaration of variables, allowing users to store data temporarily for processing. It also provides various data types, including numeric, string, date/time, and binary types.

3. **Control-of-Flow Language:**
   T-SQL includes control-of-flow statements, such as IF...ELSE, WHILE, GOTO, and BEGIN...END, allowing for conditional execution and looping, which can enhance the logic of SQL scripts.

4. **Error Handling:**
   T-SQL offers robust error handling capabilities through TRY...CATCH blocks, enabling developers to manage exceptions effectively and maintain the stability of their applications.

5. **Stored Procedures and Functions:**
   T-SQL allows for the creation of stored procedures and user-defined functions, encapsulating business logic and promoting code reuse. These can be called within other T-SQL statements, providing modularity in database applications.

6. **Triggers:**
   T-SQL supports triggers, which are special types of stored procedures that automatically execute in response to certain events on a table, such as INSERT, UPDATE, or DELETE. This enables automatic auditing, validation, or cascading actions.

7. **Cursors:**
   T-SQL provides cursors for row-by-row processing of query results, allowing for complex operations that cannot be accomplished with standard set-based SQL operations.

8. **Rich Built-in Functions:**
   T-SQL includes a variety of built-in functions for string manipulation, date/time operations, mathematical calculations, and aggregate functions, enhancing its capability for data processing and analysis.

9. **Common Table Expressions (CTEs):**
   T-SQL supports CTEs, which provide a way to write recursive queries and improve the readability of complex queries.

10. **Integration with SQL Server Features:**
    T-SQL is deeply integrated with SQL Server features, such as transactions, locking mechanisms, and indexing, allowing for optimized performance and data integrity.

Use Cases:

- **Data Analysis:** T-SQL can be used to perform complex queries, aggregations, and reporting.
- **Application Development:** Developers can use T-SQL to build applications that require database interactions.
- **Database Management:** Database administrators utilize T-SQL for managing databases, backups, and performance tuning.

Today's Lab Concepts in T-SQL

1. **Variables in T-SQL**
2. **Constants and literals in T-SQL**
3. **Hello world in T-SQL**
4. **If-Else in T-SQL**
5. **Switch (CASE) in T-SQL**
6. **Exit loop in T-SQL**
7. **While loop in T-SQL**
8. **For loop (simulated) in T-SQL**
9. **Continue in T-SQL**
10. **Loop label in T-SQL**
11. **Goto in T-SQL**
12. **Stored procedure in T-SQL**
13. **Function in T-SQL**
14. **Triggers in T-SQL**

First, we will study what are Stored procedures and how to crate user defined procedures in SQL server. Let's First Create Database and Tables.

```
CREATE TABLE tblEmployee (
id INTEGER NOT NULL,
firstName VARCHAR(255) NOT NULL,
lastName VARCHAR(255) NOT NULL,
salary INTEGER NOT NULL,
departID INTEGER NULL
);

CREATE TABLE tblDepartment (
id INTEGER NOT NULL,
name VARCHAR(255) NOT NULL,
location VARCHAR(255) NOT NULL
);
```

Now add at least 10 rows in each table above.

Stored Procedures

A SQL stored procedure (SP) is a collection SQL statements and SQL command logic, which is compiled and stored in the database. Stored procedures in SQL allows us to create SQL queries to be stored and executed on the server. Stored procedures can also be cached and reused. The main purpose of stored procedures to hide direct SQL queries from the code and improve performance of database operations such as select, update, and delete data.

User Defined Stored Procedures

User defined stored procedures are created by database developers or database administrators. These SPs contains one or more SQL statements to select, update, or delete records from database tables. User defined stored procedure can take input parameters and return output parameters. User defined stored procedure is mixture of DDL (Data Definition Language) and DML (Data Manipulation Language) commands.

Now let's create simple procedure name "ShowEmployeeTable" which will display all the values in the table.

```
Create PROCEDURE ShowEmployeeTable
AS
BEGIN

select*from tblEmployee;

END
```

Now let's execute this procedure by using keyword **"EXEC"** as shown below

```
EXEC ShowEmployeeTable;
```

90 %

Results | Messages

	id	firstName	lastName	salary	departID
1	1	Sami	Ullah	37000	1
2	2	Hamid	Khan	38000	2
3	3	Aftab	Khan	39000	3
4	4	Abhishek	Raj	32000	NULL
5	5	Haji	Ullah	37000	1

Stored Procedures with One Parameter

Now let's create modify above procedure which will take on parameter and will display values where salary will be equal to the value passed as parameter.

```
Create PROCEDURE ShowEmployeeTableWithParameter @Salary integer
AS
BEGIN

select*from tblEmployee
where salary =@Salary;

END
```

Now above procedure takes parameter declared "@" as prefix. This symbol is used to identify variables in procedures. Now query will display those employees whose salary is equal to value passed as parameter by user as shown below.

```
EXEC ShowEmployeeTableWithParameter @Salary= 37000;
```

90 %

Results | Messages

	id	firstName	lastName	salary	departID
1	1	Sami	Ullah	37000	1
2	5	Haji	Ullah	37000	1

To execute procedure with parameter, as shown above you have to pass value when executing Procedure.

Note: You can delete procedure by DROP keyword.

Stored Procedures with Multiple Parameter

You can pass multiple parameters with comma separation as shown below

```
Create PROCEDURE ShowEmployeeTableWithMultipleParameter @Salary integer, @name varchar(30)
AS
BEGIN

select*from tblEmployee
where salary =@Salary AND firstName=@name;

END
```

Result is shown below

```
EXEC ShowEmployeeTableWithMultipleParameter @Salary= 37000, @name='Sami';
```

90 %

Results | Messages

	id	firstName	lastName	salary	departID
1	1	Sami	Ullah	37000	1

Stored Procedures with Output Parameter

Output variables allows a stored procedure to return values to the calling program or script.

```sql
CREATE PROCEDURE GetEmployeeCountByDepartment
    @DepartmentName NVARCHAR(50),
    @EmployeeCount INT OUTPUT
AS
BEGIN
    SELECT @EmployeeCount = COUNT(*)
    FROM Employees
    WHERE dep_name = @DepartmentName;
END;
```

Execute it:

```sql
DECLARE @Count INT;

EXEC GetEmployeeCountByDepartment @DepartmentName = 'IT', @EmployeeCount = @Count OUTPUT;

PRINT 'Employee Count: ' + CAST(@Count AS NVARCHAR);
```

Stored Procedures with Return Value

The RETURN statement can be used to return a single integer value from a stored procedure. This is often used to indicate a success or failure status or to return specific integer-based values.

```sql
CREATE PROCEDURE spCheckDepartmentHasEmployees
    @DepartmentName NVARCHAR(50)
AS
BEGIN
    IF EXISTS (SELECT 1 FROM Employees WHERE Department = @DepartmentName)
        RETURN 1;  -- Employees found
    ELSE
        RETURN 0;  -- No employees found
END;
```

Executing the SP:

```sql
DECLARE @Result INT;

EXEC @Result = spCheckDepartmentHasEmployees @DepartmentName = 'Marketing';

IF @Result = 1
    PRINT 'Employees found in the department.';
ELSE
    PRINT 'No employees found in the department.';
```

Triggers In SQL

SQL Server triggers are special stored procedures that are executed automatically in response to the database object, database, and server events.

There are two types of Triggers:

1. DDL Trigger
2. DML Trigger

DDL Trigger

The DDL triggers are fired in response to DDL (Data Definition Language) command events that start with Create, Alter and Drop, such as Create_table, Create_view, drop_table, Drop_view and Alter_table. DDL triggers can't be used as INSTEAD OF triggers.

For example, we want to make safety trigger such that user cannot create, delete or alter any table in the database. For that purpose, we write a trigger as shown below

```
create trigger saftey
on database
for
CREATE_TABLE,
    ALTER_TABLE,
    DROP_TABLE
as

BEGIN
print'you can not create ,drop and alter table in this database'

END
```

Now whenever user tries to create table, user will be shown with the error as mentioned in above trigger. Now let's check our trigger

```
create table dummy (sample varchar(30));
```

```
90 %
Messages
you can not create ,drop and alter table in this database

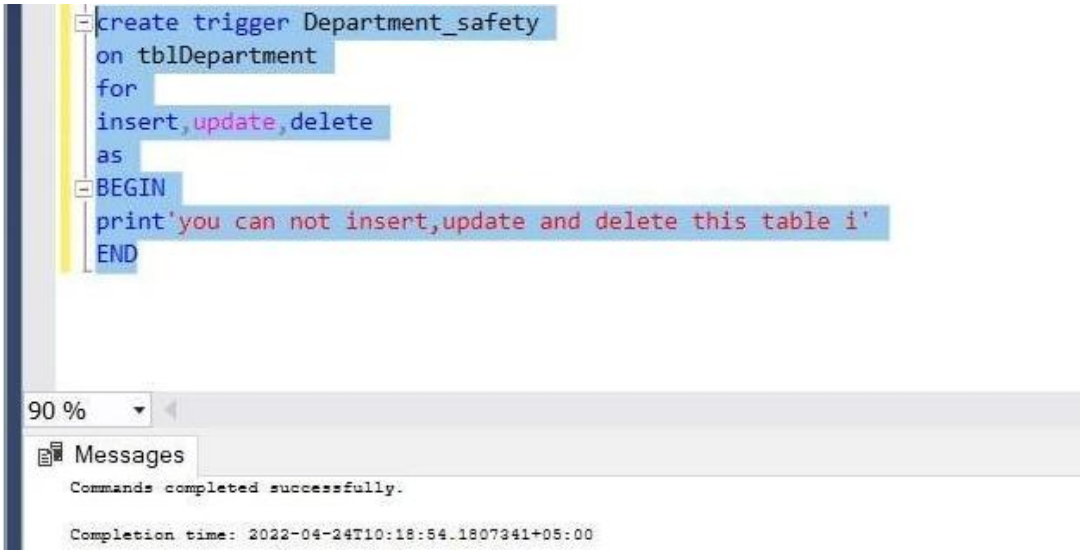
Completion time: 2022-04-24T10:11:08.0114823+05:00
```

Above screenshot shows us with error when trying to create table in our database. This trigger does not actually prevent CREATE, ALTER, or DROP operations; it only displays a message. To truly block the operation, you must use ROLLBACK statement.

DML Trigger

The DML triggers are fired in response to DML (Data Manipulation Language) command events that start with Insert, Update, and Delete. Like Insert_table, Update_view and Delete_table.

Below is the trigger defined to prevent any operation on Department table.



Now let's test our trigger as shown below

```
create trigger Department_safety
on tblDepartment
for
insert,update,delete
as
BEGIN
print'you can not insert,update and delete this table i'
END

INSERT into tblDepartment VALUES (
10, 'DUMMY', 'FICTION'
);
```

```
90 %
Messages
you can not insert,update and delete this table i

(1 row affected)

Completion time: 2022-04-24T10:20:57.2679394+05:00
```

Note as mentioned in definition, it is triggered in response to DML command. That is why command has been also executed and data has been inserted. And at the same time message has been also displayed. To prevent command execution, we have further two types of **DML triggers**

AFTER Trigger

AFTER triggers are executed after the action of an INSERT, UPDATE, or DELETE statement.

```
create trigger Department_AFTER
on tblDepartment
AFTER insert
as
BEGIN
print'Value has been inserted into table'
END
```

Above trigger is executed after insertion query.

INSTEAD of Trigger

It will tell the database engine to execute the trigger instead of executing the statement.
For example, below we have defined instead of trigger.

```
create trigger Department_INSTEAD
on tblDepartment
Instead of insert
as
BEGIN
print'Instead of trigger is activated'
END
```

Now let's test our trigger

```
INSERT into tblDepartment VALUES (
10, 'DUMMY', 'FICTION'
);
```

```
90 %
Messages
Instead of trigger is activated
```

CREATING FUNCTIONS IN SQL

Let's create simple function that takes integer value as argument and returns string.

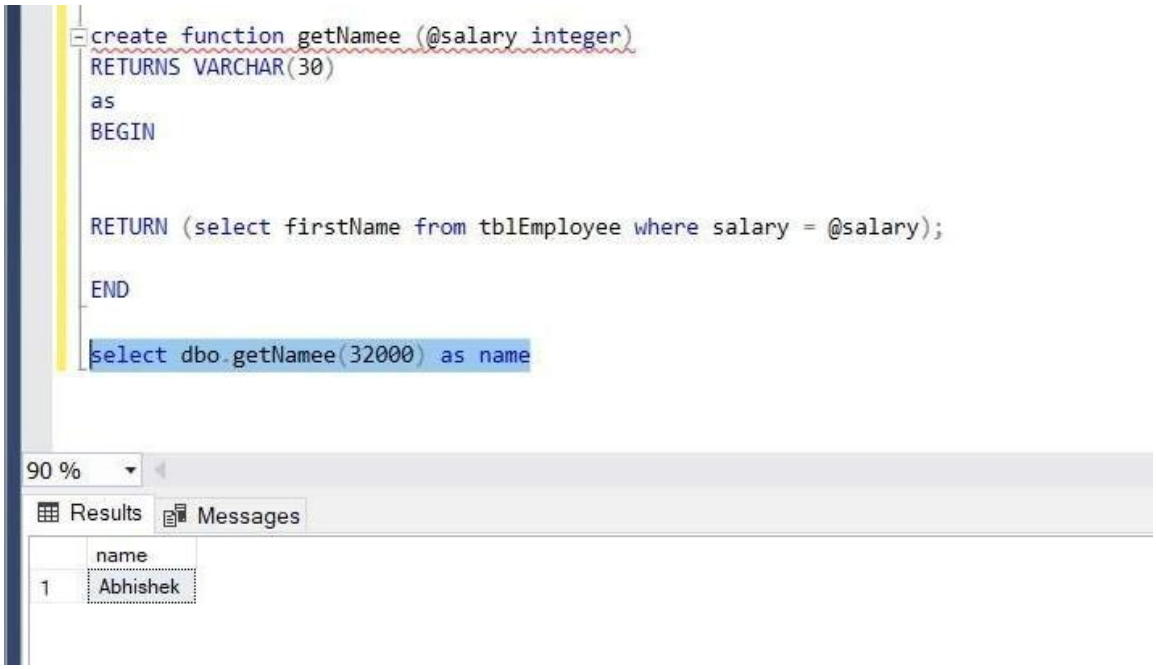
Below is the example of function that takes salary as argument and returns the name of the person who has salary equal to the passed value.

```
create function getNamee (@salary integer)
RETURNS VARCHAR(30)
as
BEGIN


RETURN (select firstName from tblEmployee where salary = @salary);

END
```

Now let's execute the function to test the result.



TASK (using NORTHWIND)

- Create a stored procedure that accepts a **country name** as a parameter and returns all customers from that country.
- Create a stored procedure that takes a **minimum order date** as a (an optional) parameter and returns all orders placed on or after that date.
- Create a **DDL trigger** that logs a message into a log table **(First create log table with two columns (Event Type, Event Description).** Now whenever a table is created in the Northwind database, an entry should be inserted into the log table capturing the event type and event description.
- Create an **AFTER INSERT DML trigger** on the `Orders` table that logs an entry in a custom table whenever a new order is added. (Do explore **Inserted** in triggers for this purpose)
- Create a function that takes an **Order ID** as a parameter and returns the **total amount** for that order (calculated as `UnitPrice * Quantity` for each item).